

```

import React, { useState, useEffect, useRef } from 'react';
import { Download, Info } from 'lucide-react';

const HelixExtractionTool = () => {
  const canvasRef = useRef(null);
  const analysisCanvasRef = useRef(null);
  const [resolution, setResolution] = useState(400);
  const [yMax, setYMax] = useState(60);
  const [computing, setComputing] = useState(false);
  const [analysisResults, setAnalysisResults] = useState(null);
  const [sliceWidth, setSliceWidth] = useState(0.1); // Width around critical lin
  const [peakThreshold, setPeakThreshold] = useState(85); // Percentile for peak

  // Known Riemann zeros
  const riemannZeros = [14.134725, 21.022040, 25.010858, 30.424876, 32.935062,
    37.586178, 40.918719, 43.327073, 48.005150, 49.773832,
    52.970321, 56.446247, 59.347044];

  const vonMangoldt = (n) => {
    if (n < 2) return 0;
    for (let p = 2; p <= Math.sqrt(n); p++) {
      let pk = p;
      while (pk <= n) {
        if (Math.abs(pk - n) < 0.001) return Math.log(p);
        pk *= p;
      }
    }
    let isPrime = n > 1;
    for (let i = 2; i <= Math.sqrt(n); i++) {
      if (n % i === 0) {
        isPrime = false;
        break;
      }
    }
    return isPrime ? Math.log(n) : 0;
  };

  const S_314 = (z) => {
    const T = Math.abs(z.im);
    if (T < 1) return 1.0;

    let sum = 0;
    const nMax = Math.floor(T) + 10;
    for (let n = 2; n <= nMax; n++) {

```

```

    const lambda = vonMangoldt(n);
    if (lambda > 0) {
        sum += lambda * Math.sin(T * Math.log(n)) / Math.sqrt(n);
    }
}
return 2.5 + 0.5 * Math.sin(2 * Math.PI * T / 10) + 0.3 * sum;
};

const Psi = (z) => {
    const s_mag = S_314(z);
    const magnitude = Math.sqrt(z.re * z.re + z.im * z.im);
    return s_mag * magnitude * Math.sin(Math.PI * z.re);
};

// Extract peaks from the data itself
const extractHelixFromData = (heatmapData, width, height) => {
    const criticalX = 0.5;
    const sliceWidthPixels = Math.floor(sliceWidth * width);
    const xStart = Math.floor((criticalX - sliceWidth/2) * width);
    const xEnd = Math.floor((criticalX + sliceWidth/2) * width);

    // For each y-position, find the brightest x-position within the slice
    const leftPeaks = [];
    const rightPeaks = [];

    for (let j = 0; j < height; j++) {
        const y = (height - j) / height * yMax;

        // Find peaks on left and right of critical line
        let maxLeftVal = 0;
        let maxLeftX = 0;
        let maxRightVal = 0;
        let maxRightX = 0;

        const centerPixel = Math.floor(criticalX * width);

        // Search left side
        for (let i = xStart; i < centerPixel; i++) {
            const val = heatmapData[j * width + i];
            if (val > maxLeftVal) {
                maxLeftVal = val;
                maxLeftX = i / width;
            }
        }

        // Search right side
        for (let i = centerPixel; i <= xEnd; i++) {

```

```

    const val = heatmapData[j * width + i];
    if (val > maxRightVal) {
      maxRightVal = val;
      maxRightX = i / width;
    }
  }

  // Only record if above threshold
  const threshold = heatmapData.reduce((a, b) => a + b, 0) / heatmapData.leng

  if (maxLeftVal > threshold) {
    leftPeaks.push({ x: maxLeftX, y: y, intensity: maxLeftVal });
  }
  if (maxRightVal > threshold) {
    rightPeaks.push({ x: maxRightX, y: y, intensity: maxRightVal });
  }
}

return { leftPeaks, rightPeaks };
};

// Measure helical parameters
const measureHelixParameters = (leftPeaks, rightPeaks, width) => {
  if (leftPeaks.length < 10 || rightPeaks.length < 10) {
    return null;
  }

  // Measure amplitude (average distance from critical line)
  const leftAmplitudes = leftPeaks.map(p => Math.abs(p.x - 0.5));
  const rightAmplitudes = rightPeaks.map(p => Math.abs(p.x - 0.5));

  const avgAmplitude = [...leftAmplitudes, ...rightAmplitudes].reduce((a, b) =>
    (leftAmplitudes.length + rightAmplitudes.length);
  const avgAmplitudePixels = avgAmplitude * width;

  // Find crossings (where left and right peaks are at same y)
  const crossings = [];
  for (let i = 0; i < Math.min(leftPeaks.length, rightPeaks.length); i++) {
    const yDiff = Math.abs(leftPeaks[i].y - rightPeaks[i].y);
    if (yDiff < 1.0) { // Within 1 unit
      // Check if they're crossing (x values close)
      const xDiff = Math.abs(leftPeaks[i].x - rightPeaks[i].x);
      if (xDiff < avgAmplitude * 0.5) {
        crossings.push((leftPeaks[i].y + rightPeaks[i].y) / 2);
      }
    }
  }
}

```

```

// Measure pitch (distance between crossings)
let avgPitch = 0;
if (crossings.length > 1) {
  const pitches = [];
  for (let i = 1; i < crossings.length; i++) {
    pitches.push(crossings[i] - crossings[i-1]);
  }
  avgPitch = pitches.reduce((a, b) => a + b, 0) / pitches.length;
}

const turns = avgPitch > 0 ? yMax / avgPitch : 0;

// Calculate frequency
const frequency = turns > 0 ? (2 * Math.PI * turns) / yMax : 0;

return {
  amplitude: avgAmplitude,
  amplitudePixels: avgAmplitudePixels,
  pitch: avgPitch,
  turns: turns,
  frequency: frequency,
  crossings: crossings.length,
  totalPeaks: leftPeaks.length + rightPeaks.length
};
};

const computeVisualization = () => {
  setComputing(true);

  setTimeout(() => {
    const canvas = canvasRef.current;
    const analysisCanvas = analysisCanvasRef.current;
    if (!canvas || !analysisCanvas) return;

    const ctx = canvas.getContext('2d');
    const analysisCtx = analysisCanvas.getContext('2d');
    const width = canvas.width;
    const height = canvas.height;

    // Generate heatmap
    const imageData = ctx.createImageData(width, height);
    const data = imageData.data;
    let maxVal = 0;
    const values = new Float32Array(width * height);

    for (let j = 0; j < height; j++) {

```

```

    for (let i = 0; i < width; i++) {
        const x = i / width;
        const y = (height - j) / height * yMax;

        const z = { re: x, im: y };
        const piEnergy = Math.PI * Math.sqrt(x * x + y * y);
        const psiEnergy = Math.abs(Psi(z));
        const diff = Math.abs(piEnergy - psiEnergy);
        const invDiff = 1.0 / (diff + 0.1);

        values[j * width + i] = invDiff;
        if (invDiff > maxVal) maxVal = invDiff;
    }
}

// Render heatmap
for (let idx = 0; idx < values.length; idx++) {
    const val = values[idx] / maxVal;
    const i = idx % width;
    const j = Math.floor(idx / width);
    const pixelIndex = (j * width + i) * 4;

    const r = Math.floor(255 * Math.pow(val, 0.5));
    const g = Math.floor(255 * Math.pow(val, 2));
    const b = Math.floor(255 * (0.5 + 0.5 * Math.sin(val * Math.PI)));

    data[pixelIndex] = r;
    data[pixelIndex + 1] = g;
    data[pixelIndex + 2] = b;
    data[pixelIndex + 3] = 255;
}

ctx.putImageData(imageData, 0, 0);

// Draw critical line
const criticalX = 0.5 * width;
ctx.strokeStyle = 'orange';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(criticalX, 0);
ctx.lineTo(criticalX, height);
ctx.stroke();

// Draw analysis region
const sliceWidthPixels = sliceWidth * width;
ctx.strokeStyle = 'rgba(255, 255, 0, 0.3)';
ctx.lineWidth = 1;

```

```

ctx.strokeRect(criticalX - sliceWidthPixels/2, 0, sliceWidthPixels, height)

// Extract helix from data
const { leftPeaks, rightPeaks } = extractHelixFromData(values, width, height)

// Draw extracted peaks on main canvas
ctx.fillStyle = 'cyan';
leftPeaks.forEach(peak => {
  const px = peak.x * width;
  const py = height - (peak.y / yMax) * height;
  ctx.beginPath();
  ctx.arc(px, py, 2, 0, 2 * Math.PI);
  ctx.fill();
});

ctx.fillStyle = 'magenta';
rightPeaks.forEach(peak => {
  const px = peak.x * width;
  const py = height - (peak.y / yMax) * height;
  ctx.beginPath();
  ctx.arc(px, py, 2, 0, 2 * Math.PI);
  ctx.fill();
});

// Draw known zeros
ctx.fillStyle = 'lime';
riemannZeros.forEach(zero => {
  const y = height - (zero / yMax) * height;
  if (y >= 0 && y <= height) {
    ctx.beginPath();
    ctx.arc(criticalX, y, 4, 0, 2 * Math.PI);
    ctx.fill();
  }
});

// ANALYSIS CANVAS - Show extracted geometry
analysisCtx.fillStyle = '#000';
analysisCtx.fillRect(0, 0, analysisCanvas.width, analysisCanvas.height);

// Draw extracted peaks as a clean view
analysisCtx.strokeStyle = 'cyan';
analysisCtx.lineWidth = 2;
analysisCtx.beginPath();
leftPeaks.forEach((peak, idx) => {
  const px = peak.x * width;
  const py = height - (peak.y / yMax) * height;
  if (idx === 0) analysisCtx.moveTo(px, py);

```

```

        else analysisCtx.lineTo(px, py);
    });
    analysisCtx.stroke();

    analysisCtx.strokeStyle = 'magenta';
    analysisCtx.beginPath();
    rightPeaks.forEach((peak, idx) => {
        const px = peak.x * width;
        const py = height - (peak.y / yMax) * height;
        if (idx === 0) analysisCtx.moveTo(px, py);
        else analysisCtx.lineTo(px, py);
    });
    analysisCtx.stroke();

    // Draw critical line on analysis
    analysisCtx.strokeStyle = 'orange';
    analysisCtx.lineWidth = 1;
    analysisCtx.beginPath();
    analysisCtx.moveTo(criticalX, 0);
    analysisCtx.lineTo(criticalX, height);
    analysisCtx.stroke();

    // Measure parameters
    const params = measureHelixParameters(leftPeaks, rightPeaks, width);
    setAnalysisResults(params);

    setComputing(false);
}, 100);
};

useEffect(() => {
    computeVisualization();
}, [resolution, yMax, sliceWidth, peakThreshold]);

return (
    <div className="w-full min-h-screen bg-gray-900 text-white p-6">
        <div className="max-w-7xl mx-auto">
            <h1 className="text-3xl font-bold mb-2">Data-Driven Helix Extraction Tool
            <p className="text-gray-400 mb-6">
                Measuring the helical structure directly from the heatmap data, not imp
            </p>

            <div className="grid grid-cols-1 xl:grid-cols-4 gap-6">
                { /* Visualizations */ }
                <div className="xl:col-span-3 space-y-4">
                    { /* Original Heatmap with Detected Peaks */ }
                    <div className="bg-gray-800 rounded-lg p-4">

```

```

    <h3 className="text-lg font-semibold mb-2">Original Data + Detected
    <p className="text-xs text-gray-400 mb-2">Cyan/Magenta dots show pe
    <canvas
      ref={canvasRef}
      width={600}
      height={800}
      className="w-full border border-gray-700 rounded"
    />
  </div>

  {/* Extracted Geometry */}
  <div className="bg-gray-800 rounded-lg p-4">
    <h3 className="text-lg font-semibold mb-2">Extracted Helix Geometry
    <p className="text-xs text-gray-400 mb-2">Connected peaks showing t
    <canvas
      ref={analysisCanvasRef}
      width={600}
      height={800}
      className="w-full border border-gray-700 rounded bg-black"
    />
  </div>
</div>

{/* Controls and Results */}
<div className="space-y-4">
  <div className="bg-gray-800 rounded-lg p-4">
    <h3 className="text-lg font-semibold mb-3">Extraction Controls</h3>

    <div className="space-y-4">
      <div>
        <label className="block text-sm mb-2">
          Y-axis Range: {yMax}
        </label>
        <input
          type="range"
          min="20"
          max="100"
          value={yMax}
          onChange={ (e) => setYMax (Number (e.target.value)) }
          className="w-full"
        />
      </div>

      <div>
        <label className="block text-sm mb-2">
          Analysis Width: {sliceWidth.toFixed(2)}
        </label>

```



```

        <input
            type="range"
            min="0.05"
            max="0.3"
            step="0.01"
            value={sliceWidth}
            onChange={(e) => setSliceWidth(Number(e.target.value))}
            className="w-full"
        />
        <p className="text-xs text-gray-400 mt-1">Width of region to an
    </div>
</div>
</div>

{/* Analysis Results */}
<div className="bg-blue-900 bg-opacity-30 rounded-lg p-4 border borde
    <div className="flex items-start space-x-2 mb-3">
        <Info className="w-5 h-5 text-blue-400 mt-0.5 flex-shrink-0" />
        <h3 className="text-lg font-semibold text-blue-300">Measured Para
    </div>

    {computing ? (
        <p className="text-yellow-400 text-sm">Computing...</p>
    ) : analysisResults ? (
        <div className="space-y-2 text-sm">
            <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
                <div className="text-gray-400">Amplitude (from center)</div>
                <div className="text-xl font-bold text-cyan-400">
                    {analysisResults.amplitudePixels.toFixed(1)}px
                </div>
                <div className="text-xs text-gray-500">
                    ({(analysisResults.amplitude * 100).toFixed(1)}% of width)
                </div>
            </div>

            <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
                <div className="text-gray-400">Helical Pitch</div>
                <div className="text-xl font-bold text-magenta-400">
                    {analysisResults.pitch > 0 ? analysisResults.pitch.toFixed(
                </div>
                <div className="text-xs text-gray-500">
                    Imaginary units per rotation
                </div>
            </div>

            <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
                <div className="text-gray-400">Complete Turns</div>

```

```

        <div className="text-xl font-bold text-green-400">
            {analysisResults.turns > 0 ? analysisResults.turns.toFixed(
        </div>
        <div className="text-xs text-gray-500">
            Over Y-range of {yMax}
        </div>
    </div>

    <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
        <div className="text-gray-400">Spatial Frequency (k)</div>
        <div className="text-xl font-bold text-yellow-400">
            {analysisResults.frequency > 0 ? analysisResults.frequency.
        </div>
        <div className="text-xs text-gray-500">
            Compare to Gemini's k=0.63
        </div>
    </div>

    <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
        <div className="text-gray-400">Detected Crossings</div>
        <div className="text-xl font-bold text-orange-400">
            {analysisResults.crossings}
        </div>
    </div>

    <div className="bg-gray-800 bg-opacity-50 p-2 rounded">
        <div className="text-gray-400">Total Peaks Found</div>
        <div className="text-xl font-bold">
            {analysisResults.totalPeaks}
        </div>
    </div>
    </div>
    ) : (
        <p className="text-gray-400 text-sm">No analysis yet</p>
    )}
</div>

<div className="bg-gray-800 rounded-lg p-4">
    <h3 className="text-lg font-semibold mb-3">Method</h3>
    <div className="space-y-2 text-xs text-gray-300">
        <p>1. Extract peak brightness in narrow band around  $\text{Re}(z)=0.5$ </p>
        <p>2. Separate left vs right peaks (two strands)</p>
        <p>3. Measure actual amplitude from detected positions</p>
        <p>4. Find crossings where strands meet</p>
        <p>5. Calculate pitch, turns, and frequency from data</p>
        <p className="text-yellow-400 mt-3">
            This measures what's actually in the data, not what theory pred

```

```
        </p>
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};

export default HelixExtractionTool;
```